

TÀI LIỆU DEEP DIVE: POLICY GRADIENT VÀ THUẬT TOÁN REINFORCE

1. Giải thích cơ chế cập nhật trọng số: $\theta \leftarrow \theta + \alpha \nabla J(\theta)$

Trong Học máy truyền thống, chúng ta thường sử dụng thuật toán Gradient Descent (hạ gradient) để tìm cách giảm thiểu hàm mất mát (Loss Function). Khi đó, trọng số sẽ được cập nhật bằng cách trừ đi gradient.

Tuy nhiên, trong bài toán Reinforcement Learning (RL), hàm mục tiêu $J(\theta)$ lại biểu diễn cho **Tổng phần thưởng kỳ vọng (Expected Return)**. Do mục đích của Agent (tác tử) là thu thập được càng nhiều điểm càng tốt, bài toán của chúng ta là bài toán **Cực đại hóa (Maximization)**. Vì vậy, ta phải sử dụng thuật toán **Gradient Ascent (Tối ưu hóa dốc lên)**.

Cơ chế cập nhật trọng số diễn ra như sau:

- **θ (Theta):** Là tập hợp các tham số (trọng số) của mạng nơ-ron đóng vai trò là Policy (mạng quyết định hành động).
- **α (Alpha):** Là hệ số học (Learning Rate), quyết định bước nhảy của thuật toán mỗi lần cập nhật là lớn hay nhỏ.
- **$\nabla J(\theta)$ (Gradient của hàm mục tiêu):** Là vector chỉ ra hướng thay đổi của tham số θ sao cho hàm mục tiêu $J(\theta)$ tăng nhanh nhất.
- **Dấu cộng (+):** Ta lấy trọng số hiện tại "cộng" thêm một lượng gradient để mạng nơ-ron dịch chuyển theo hướng làm tăng phần thưởng (đi lên đỉnh).

(Lưu ý: Khi áp dụng vào thực tế với thư viện PyTorch, do PyTorch chỉ mặc định hỗ trợ hạ Gradient, Nhóm sẽ đổi dấu hàm mục tiêu thành $Loss = -J(\theta)$ để tái tạo lại cơ chế Gradient Ascent này).

2. Công thức Gradient Estimate cho thuật toán REINFORCE

Để lập trình thuật toán, chúng ta không thể dùng công thức kỳ vọng lý thuyết, mà phải dùng một giá trị ước lượng dựa trên các mẫu hành động thực tế (Gradient Estimate). Ký hiệu là $\hat{g}$.

Áp dụng Log-derivative Trick (Mẹo đạo hàm logarit) và tính chất Causality (hành động hiện tại không làm thay đổi phần thưởng quá khứ), ta có công thức Gradient Estimate dùng trong 1 Episode như sau:

$$\hat{g} = \sum \nabla \log \pi(a_t | s_t) G_t \text{ (với } t \text{ chạy từ } 0 \text{ đến } T)$$

Trong đó:

- **$\nabla \log \pi(a_t | s_t)$:** Là đạo hàm logarit của xác suất Agent chọn hành động a_t khi đang ở trạng thái s_t .

- G_t : Là phần thưởng tích lũy (Return) tính từ thời điểm bước t cho đến khi kết thúc ván chơi.

Bản chất toán học của công thức: Nếu chuỗi hành động mang lại phần thưởng G_t lớn (hành động tốt), giá trị gradient này sẽ lớn, làm "tăng mạnh" xác suất chọn lại hành động đó trong tương lai. Ngược lại, nếu G_t nhỏ hoặc âm, mạng nơ-ron sẽ tự động "giảm" xác suất của hành động đó đi.

3. So sánh sự khác biệt giữa Policy Gradient (PG) và Value-based (DQN)

Để củng cố lý do nhóm lựa chọn thuật toán Policy Gradient thay vì DQN, dưới đây là đối chiếu toán học giữa hai phương pháp:

- **Về mục tiêu tối ưu:** DQN tối ưu hóa hàm giá trị $Q(s,a)$, sau đó dùng hàm argmax để chọn hành động có giá trị Q lớn nhất. Trong khi đó, PG (REINFORCE) tối ưu hóa trực tiếp mạng Policy $\pi(a|s)$ để trả ra thẳng phân phối xác suất của các hành động.
- **Về tính chất hành động:** DQN dễ bị kẹt ở Deterministic Policy (tại một trạng thái luôn đưa ra một hành động cố định, thiếu sự linh hoạt). PG sử dụng Stochastic Policy (hành động ngẫu nhiên theo tỷ lệ xác suất) nên có khả năng khám phá (exploration) môi trường mượt mà và tự nhiên hơn.
- **Về không gian hành động:** DQN chỉ giải quyết tốt bài toán có số lượng hành động rời rạc (Discrete). PG có khả năng mở rộng để xử lý không gian hành động liên tục (Continuous) một cách dễ dàng.
- **Về sự hội tụ:** DQN thường bị mất ổn định và dao động mạnh vì phải xấp xỉ giá trị Q liên tục. PG có tính hội tụ (Convergence) cực kỳ ổn định và mượt mà. Tuy nhiên, điểm yếu của PG là **phương sai (Variance) cao**. Để khắc phục, Nhóm 05 đã sử dụng kỹ thuật "Trò đi đường cơ sở" (Baseline) trong code thực tế để giảm phương sai này.